

第六章：分支限界算法

Design and Analysis of Computer Algorithms

Xiu-Jun GONG

College of Intelligence and Computing, Tianjin University

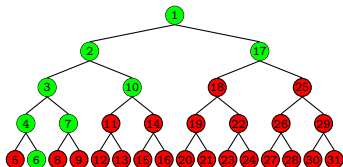
Email: gongxj@tju.edu.cn

April, 2025

- 1 简介
 - 动机
 - 什么是分支定界法
 - 与其他算法比较
- 2 带罚款额的作业调度问题
- 3 旅行商问题
- 4 A* 算法

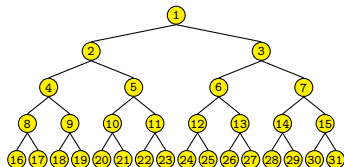
状态空间搜索：DFS 与 BFS

考虑 0/1 背包问题，其状态空间可以通过定长元组表示进行展开
 $(n=4, c=7, p=[4, 7, 9, 10], w=[1, 2, 3, 5])$:



使用深度优先搜索 (DFS):

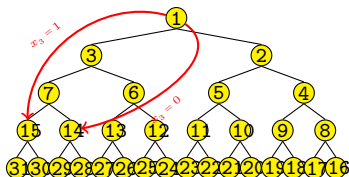
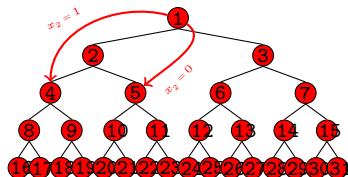
- 扩展节点每次生成一个子节点
- 需要 $O(n)$ 空间存储活节点
- **限界**将节点数从 31 减少到 8



使用广度优先搜索 (BFS):

- 扩展节点每次生成所有子节点
- 需要 $O(2^n)$ 空间存储活节点
- 如何**管理**活节点？如何**限界**？

活节点管理



- FIFO 队列：活节点存储在队列
- LIFO 栈：活节点存储在栈
- 最大收益（最小成本）：活节点存储在最大/最小堆

需要限界解状态，使得最优候选解 能够被快速发现，同时无效候选 被及时丢弃。如何实现？

解状态限界

已知贪心算法可以求解部分背包问题的最优值。

部分松弛法

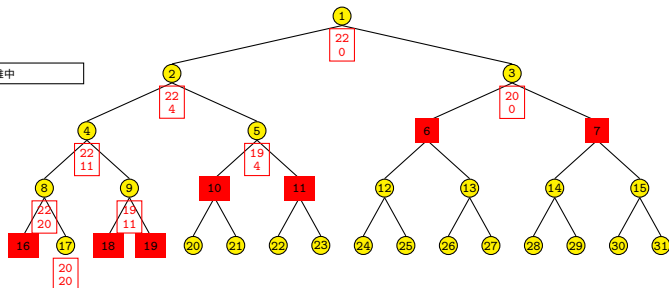
部分背包问题的最优值是 0/1 背包问题的更紧上界

- 可以为每个子问题（对应解状态）计算上界
- 其下界对应当前装入方案的价值
- 上下界可以指导最优解的搜索：
 - ① 对于任何状态，其下界应不大于上界
 - ② 对于状态 A 和 B，若 A 的上界不大于 B 的下界，则 A 应被丢弃

基于上下界的 BFS 搜索

实例: $n=4, c=7, p=[4, 7, 9, 10], w=[1, 2, 3, 5]$

活节点按上界升序存储在最大堆中



- 节点 1 的上界 $= 4 + 7 + 9 + (7 - 1 - 2 - 3) * (10 / 5) = 22$, 下界 $= 0$
- 节点 2 的上界 $= 4 + 7 + 9 + (7 - 1 - 2 - 3) * (10 / 5) = 22$, 下界 $= 4$
- 节点 3 的上界 $= 7 + 9 + (7 - 2 - 3) * (10 / 5) = 20$, 下界 $= 0$

定义

分支定界算法 (B&B) 是解决**最优化**问题的系统方法，特别适用于组合优化和整数规划问题。通过状态空间，**划分**问题为更小的子问题（分支），使用**边界**评估潜力，并丢弃不可能产生最优解的子问题（剪枝），从而高效探索解空间。

- ① **分支:**
 - 将原问题分解为更小的子问题
- ② **限界:**
 - 为每个子问题计算**边界**（最大化问题上界，最小化问题下界）
- ③ **当前最优解:**
 - 目前找到的最佳可行解。当发现更优解时更新
- ④ **剪枝:**
 - 若子问题的边界劣于当前最优解，则丢弃该子问题

算法步骤

- ① 初始化:
 - 将原问题加入活动子问题列表
 - 可选用启发式方法获取初始当前最优解
- ② 选择:
 - 选择下一个待探索子问题（如基于边界的最优优先搜索）
- ③ 分支:
 - 将选定子问题分解为更小子问题
- ④ 限界:
 - 计算新子问题的边界，剪除边界劣于当前最优解的子问题
- ⑤ 更新:
 - 若子问题产生优于当前最优解的可行解，则更新当前最优解
- ⑥ 终止:
 - 重复直至无活动子问题。当前最优解即为最优解

伪代码

- ① 初始化:
 - 优先队列 Q (最小化用最小堆, 最大化用最大堆)
 - 添加根节点 (含初始状态和边界计算)
 - $\text{best_solution} \leftarrow \text{None}$
 - $\text{best_value} \leftarrow \infty$ (最小化) 或 $-\infty$ (最大化)
- ② 当 Q 非空时:
 - a. 节点 $\leftarrow$ 从 Q 中提取最有希望的边界节点
 - b. 若节点边界劣于 best_value : 剪枝
 - c. 若节点是完整解:
 - 若优于当前最优解, 则更新 best_value 和 best_solution
 - 继续循环
 - d. 分支: 通过添加下一个决策生成子节点
 - e. 对每个子节点:
 - 计算子节点边界
 - 若子节点边界优于 best_value : 将子节点加入 Q
- ③ 返回 $\text{best_solution}, \text{best_value}$

优势与挑战

- **优势:**

- 通过剪枝避免穷举搜索
- 适用于多种优化问题的灵活框架
- 可集成启发式方法加速收敛

- **挑战:**

- 效果依赖边界质量
- 大规模问题内存消耗大
- 最坏情况复杂度仍为指数级

- **效率优化策略**

- **紧致边界:** 利用问题特性计算精确边界
- **分支顺序:** 优先处理对目标影响最大的变量

B&B 与回溯法

① 核心目标

- 分支限界法: 解决**最优化问题**
- 回溯法: 解决**决策/搜索问题**

② 实现方式

- B&B: 使用**边界**剪除无法改进当前最优解的路径
- 回溯法: 仅在违反约束时剪枝

③ 内存与效率

- B&B: 内存密集, 高度依赖**边界质量**
- 回溯法: 内存需求较低, 可能不必要地探索更多路径

④ 最优性保证

- B&B: 保证**最优解**
- 回溯法: 找到所有解但不保证优化

Outline

- 1 简介
- 2 带罚款额的作业调度问题
 - 问题描述
 - 方法详解
- 3 旅行商问题
- 4 A^* 算法

定义

给定 n 个作业 $\{j_1, j_2, \dots, j_n\}$ 和单个处理器，每个作业 j_i 关联一个三元组 (p_i, d_i, t_i) ，其中：

- t_i : 完成作业 j_i 所需的处理时间
- d_i : 作业 j_i 必须完成的截止时间
- p_i : 若作业 j_i 未能在截止时间前完成，将产生的惩罚

目标是在处理器上调度一个作业子集 J ，以最小化所有未调度作业或超期作业的总惩罚。

关键约束：

- ① **单处理器**: 任意时刻只能处理一个作业
- ② **截止时间合规**: 若作业 j_i 被调度，其开始时间 s_i 必须满足
$$s_i + t_i \leq d_i$$
- ③ **非抢占**: 作业一旦开始不能中断

形式化表示

目标： 在处理器上调度作业子集 J 以最小化

$$\sum_i^n p_i * (1 - x_i)$$

约束条件：

- ① $x_i = 1$ 当且仅当 $j_i \in J$, 否则 $x_i = 0$
- ② 每个被调度作业 j_i 必须连续处理 t_i 时间单位, 并在截止时间前完成
- ③ $\sum_i^n t_i * x_i \leq d_k \quad (\forall k = 1, 2, \dots, n \text{ 且 } d_1 \leq d_2 \leq \dots \leq d_n)$

解决方法

- ① **贪心算法**: 按 p_i 降序排序, 采用可行性调度 (如最早截止时间优先)
- ② **动态规划**: 处理作业子集 (伪多项式时间)
- ③ **分支限界法**: 系统探索作业子集, 使用罚款额上界限界

关键洞见

满足截止时间的最优调度顺序是“**最早截止时间优先**” (EDF) 顺序。这种顺序可以最大化按时完成的作业数量, 从而最小化惩罚。

分支限界

- 分支: 设 i 为 E 节点, 将子解空间 S_i 划分为

- 子解空间 S_i^0 ($x_{i+1} = 0$)
- 子解空间 S_i^1 ($x_{i+1} = 1$)
- $S_i = S_i^0 \cup S_i^1$

- 估计子解空间的上下界

- 下界 (当前惩罚):

$$L(i+1) = \sum_{j=1}^{i+1} (1 - x_j) * p_j$$

- 上界 (最坏情况):

$$U(i+1) = L(i+1) + \sum_{j=i+2}^n p_j$$

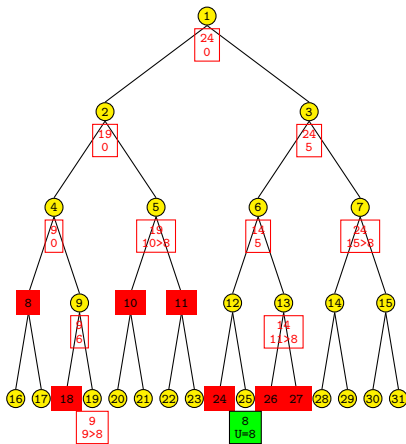
- 限界: 对每个子解空间

- 检查可行性条件: 不满足则剪枝
- 比较下界: 若大于活动节点的上界则剪枝
- 增强上界: 用 $U(i)$ 更新全局上界 U , 保持其为最小上界

示例

$n=4$ 时, $(p, d, t)_1^4 = \{(5, 1, 1), (10, 3, 2), (6, 2, 1), (3, 1, 1)\}$

步骤	堆 E	节点 (U)
0	1	1 (24)
1	2,3	2 (19)
2	4,3,5	4 (9)
3	3,9,5	3 (9)
4	6,9,5,7	6 (9)
5	12,9,5,13,7	12 (8)
6	9,25,5,13,7	9(8)
7	25,19,5,13,7	25(8)
7	19,5,13,7	终止



- 活动节点存储在最小堆中
- 按最早截止时间优先 (EDF) 顺序选择作业

- 1 简介
- 2 带罚款额的作业调度问题
- 3 旅行商问题
 - 方法
 - 规约成本矩阵
- 4 A^* 算法

关键组件

① 状态表示：

- 部分路径表示为已访问城市的序列。例如， $[1, 3, 2]$ 表示推销员迄今为止已访问了城市 $1 \rightarrow 3 \rightarrow 2$ 。

② 分支：

- 在每一步，算法通过添加一个未访问的城市来扩展当前的部分路径。这会创建多个分支，每个分支对应于下一个不同的城市。

③ 限界：

- 对于每个部分路径，计算完成路径的成本的下界。如果此下界超过最佳已知解决方案的成本，则剪枝该分支。

④ 搜索策略：

- 算法使用基于下界的**最佳优先搜索**探索状态空间树。

算法步骤

① 初始化

- 将初始部分路径设为 $[1]$ (从城市 1 开始)。
- 初始化 $UB = \infty$ 。

② 下界计算

- 最小生成树 (MST)
- 缩减成本矩阵

③ 分支

- 通过访问未访问的城市扩展当前路径。

④ 限界

- 计算每个新节点的下界。
- 如果 $LB > UB$, 则剪枝该节点。

⑤ 更新最佳解决方案

- 如有必要, 更新上界。

⑥ 重复

- 继续 $3 \rightarrow 5$ 直到所有节点都被探索或剪枝。

⑦ 输出: 返回找到的最佳路径。

定义

规约成本矩阵是从原始距离矩阵 D 中通过减去行和列的最小值派生出来的，使得每一行和每一列都包含至少一个零。这种转换确保规约矩阵的总成本是完成路径成本的有效下界。

关键洞见

- 减去 行和列的最小值 不会改变最优解，但提供了更紧的下界。
- 减去的总量给出了任何可行路径成本的有效下界。

计算规约成本矩阵的步骤

- **步骤 1: 行规约:** 对于 D 的每一行:
 - 找到该行中的最小值。
 - 从该行的每个元素中减去这个值, 记为 RM 。
 - 令 $R[i]$ 表示第 i 行的行规约数。

$$\text{行规约数} = \sum_{i=1}^n R[i]$$

- **步骤 2: 列规约:** 对于 RM 的每一列:
 - 找到该列中的最小值。
 - 从该列的每个元素中减去这个值作为 CM 。
 - 令 $C[j]$ 表示第 j 列的列规约数。

$$\text{列规约数} = \sum_{j=1}^n C[j]$$

- **步骤 3: 总下界**

$$\text{下界} = \sum_{i=1}^n R[i] + \sum_{j=1}^n C[j]$$

规约成本矩阵的例子

考虑以下 5 个城市的距离矩阵 D :

$$D = \begin{bmatrix} \infty & 20 & 30 & 10 & 11 \\ 15 & \infty & 16 & 4 & 2 \\ 3 & 5 & \infty & 2 & 4 \\ 19 & 6 & 18 & \infty & 3 \\ 16 & 4 & 7 & 16 & \infty \end{bmatrix}$$

注释:

- 行规约: $RM = D - R$
- 列规约: $CM = RM - C$
- 最终规约: $RCM = CM$
- 下界 = $\sum R + \sum C$

步骤 1: 行规约: $RM = D - R$

行规约向量: $R = [10, 2, 2, 3, 4]$

$$RM = \begin{bmatrix} \infty & 20 & 30 & 10 & 11 \\ 15 & \infty & 16 & 4 & 2 \\ 3 & 5 & \infty & 2 & 4 \\ 19 & 6 & 18 & \infty & 3 \\ 16 & 4 & 7 & 16 & \infty \end{bmatrix} - \begin{bmatrix} 10 \\ 2 \\ 2 \\ 3 \\ 4 \end{bmatrix} = \begin{bmatrix} \infty & 10 & 20 & 0 & 1 \\ 13 & \infty & 14 & 2 & 0 \\ 1 & 2 & \infty & 0 & 2 \\ 16 & 3 & 15 & \infty & 0 \\ 12 & 0 & 3 & 12 & \infty \end{bmatrix}$$

步骤 2: 列规约: $CM = RM - C$

列规约向量: $C = [1, 0, 3, 0, 0]$

$$CM = \begin{bmatrix} \infty & 10 & 20 & 0 & 1 \\ 13 & \infty & 14 & 2 & 0 \\ 1 & 2 & \infty & 0 & 2 \\ 16 & 3 & 15 & \infty & 0 \\ 12 & 0 & 3 & 12 & \infty \end{bmatrix} - \begin{bmatrix} 1 \\ 0 \\ 3 \\ 0 \\ 0 \end{bmatrix}^T = \begin{bmatrix} \infty & 10 & 17 & 0 & 1 \\ 12 & \infty & 11 & 2 & 0 \\ 0 & 3 & \infty & 0 & 2 \\ 15 & 3 & 12 & \infty & 0 \\ 11 & 0 & 0 & 12 & \infty \end{bmatrix}$$

步骤 3: 总下界

$$\text{下界} = \sum R + \sum C = 25$$

因此, 该代价矩阵的下界是 **25**。

在分支限界法中使用规约成本矩阵

步骤 1: 初始化

- 从 D 开始并计算其缩减成本矩阵以获得初始界限。

步骤 2: 分支

- 通过访问未访问的城市之一来扩展当前部分路径。为每个可能的扩展创建一个新节点。

步骤 3: 更新规约成本矩阵

- 对于每个分支, 修改 D 以反映部分路径施加的约束
- 重新计算缩减成本矩阵以获得新的界限。

步骤 4: 限界

- 如果下界超过当前最佳解决方案, 则剪枝该分支。

规约成本矩阵的优缺点

- **优点**

- **高效的下界**：提供了一个紧的下界，有助于早期剪枝许多次优分支。
- **简单性**：计算涉及基本的算术运算（查找最小值和减去值）。
- **可扩展性**：适用于中小型 TSP 实例。

- **局限性**

- **计算复杂度**：在每个节点重新计算缩减成本矩阵对于大型实例来说可能非常昂贵。
- **界限的紧密性**：可能不足以消除所有不必要的分支。
- **依赖距离矩阵**：界限的质量取决于距离矩阵的结构。

- 1 简介
- 2 带罚款额的作业调度问题
- 3 旅行商问题
- 4 **A* 算法**
 - 什么是 A* 算法?
 - 算法
 - 八数码问题

简介

- **A* 算法**于 1968 年由 Peter Hart、Nils Nilsson 和 Bertram Raphael 在斯坦福研究所正式提出，作为 Shakey the Robot 项目的一部分，这是最早由人工智能驱动的自主机器人之一。
- 它是一种基础的**路径搜索**和**图遍历**算法，广泛应用于计算机科学、人工智能、机器人和游戏开发中。
- 它受到以下启发：
 - **Dijkstra 算法**：保证找到**最短路径**，但会向**所有方向**探索。
 - **启发式搜索**：估计从当前节点到目标节点的成本。
- 它通过**启发式引导****优先选择**可能通向**目标**的路径，在保证最优性的同时显著减少计算时间。

关键概念

① A* 使用**成本函数**评估节点:

$$f(n) = g(n) + h(n)$$

- $g(n)$: 从起始节点到当前节点 n 的实际成本。
- $h(n)$: 从 n 到目标节点的启发式估计成本。
- $f(n)$: 通过 n 的路径的总估计成本。

② 启发式函数 ($h(n)$)

- **作用**: 引导搜索朝向目标, 避免不必要的探索。
- **要求**:
 - **可接受性**: 永远不会高估真实成本 ($h(n) \leq h^*(n)$)。
 - **一致性 (单调性)**: 对于所有邻居 n' 满足 $h(n) \leq \text{cost}(n, n') + h(n')$ 。

③ 数据结构

- **开放列表**: 优先探索的节点。
- **关闭列表**: 跟踪已评估的节点以避免重复。

A* 的特性

- **完整性:**
 - 如果存在解, A* 保证能找到它。
- **最优性:**
 - 只要使用的启发式函数是可接受且一致的, 它就能保证找到从起始节点到目标节点的最低成本路径。
- **效率:**
 - 一个好的启发式函数可以显著减少被探索的节点数量, 而一个差的启发式函数可能导致性能类似于无信息搜索。

算法工作流程

① 初始化:

- 将起始节点添加到开放列表中, 其中 $f(n) = g(n) + h(n)$ 。
- 初始化所有节点的 $g(n)$ 为无穷大, 除了起始节点 ($g = 0$)。

② 主循环:

- 从开放列表中提取 $f(n)$ 最小的节点。
- 如果该节点是目标节点, 则终止; 否则, 将其添加到关闭列表中。
- **扩展其邻居:**
 - 跳过关闭列表中的节点或被阻塞的节点。
 - 如果找到更便宜的路径, 则更新邻居的 $g(n)$ 和 $f(n)$ 。
 - 将新的或改进的节点添加到开放列表中。

③ 路径重建:

- 使用父指针从目标回溯到起点。

设计启发式函数

① 分析问题

- **移动规则**：沿水平、垂直或对角线移动？
- **图结构**：是网格、图还是连续空间？是否有障碍物？
- **目标状态**：终点如何定义（例如，坐标、状态）？

② 选择基础启发式函数

- **网格（4 方向移动）**：使用**曼哈顿距离**：

$$h(n) = |x_n - x_{\text{goal}}| + |y_n - y_{\text{goal}}|$$

- **网格（8 方向移动）**：使用**对角线距离**：

$$h(n) = \max(|x_n - x_{\text{goal}}|, |y_n - y_{\text{goal}}|)$$

- **连续空间**：使用**欧几里得距离**：

$$h(n) = \sqrt{(x_n - x_{\text{goal}})^2 + (y_n - y_{\text{goal}})^2}$$

③ 利用领域知识增强基础启发式函数

- 带障碍物的路径寻找：使用简化路径假设。
- 道路网络：预计算地标距离以获得更紧的界限。

④ 高级技术

- 基于学习的启发式函数：
 - 训练机器学习模型预测 $h(n)$ ，但需通过约束确保可接受性。
- 动态启发式函数：
 - 根据实时变化调整 $h(n)$ 。
- 特定领域的技巧：
 - 利用问题对称性或模式数据库。

问题描述

八数码问题是一个简单的游戏，由八个滑动瓷砖组成，编号为 1 到 8，放置在一个 3×3 的九格棋盘上。

- 其中一个单元始终为空。
- 任何相邻（水平和垂直）的瓷砖都可以移动到空单元中。
- 游戏的目标是从初始配置开始并最终达到目标配置。

$$\begin{bmatrix} 2 & 8 & 3 \\ 1 & 6 & 4 \\ 7 & \blacksquare & 5 \end{bmatrix} \Rightarrow \begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \\ 7 & 8 & \blacksquare \end{bmatrix}$$

状态空间表述

状态空间表示了 3×3 棋盘上所有可能的瓷砖和空白方块的配置。

- 每个状态是一个 3×3 矩阵，其中：
 - 1-8：具有唯一数字的瓷砖。
 - 0：表示空白空间。
- 状态转换
 - 空白空间可以向上、向下、向左或向右移动（如果有效）。
 - 每次移动都会生成一个新状态。
- 约束条件
 - 空白空间不能移出 3×3 网格。
- 成本函数
 - $f(n) = g(n) + h(n)$
 - $g(n)$ ：从初始状态到状态 n 的移动次数。
 - $h(n)$ ：启发式函数。

启发式函数设计

- 曼哈顿距离

$$h(n) = \sum_{i=1}^8 (\text{瓷砖 } i \text{ 与其目标位置的距离})$$

- 示例：如果在目标状态中第 1 块位于 (0,0)，而在当前状态中位于 (1,1)：
第 1 块的曼哈顿距离 = $|1 - 0| + |1 - 0| = 2$ 。

- 错位瓷砖数

- 不在其目标位置的瓷砖数量（不包括空白空间）。
- $h(n)$ = 不在其目标位置的瓷砖数量

- 启发式的比较

启发式函数	可接受性?	效率
曼哈顿距离	是	快速、准确
错位瓷砖数	是	简单、不够准确

Coming up: NP Complete
Problems